

SAARA KETHA HARVEST HUB · ISE_WE_0101_23

Epic 3 – Inventory, Buyer Orders & Distribution Management

Concepts, validations, demo script and viva questions

1. Epic overview
2. Core concepts
3. Validations
4. Suggested live demo
5. Likely viva questions and model answers
6. Know your limitations
7. Where to find it in the code
8. Appendix – system foundations (same in every guide)

Study guide generated from the implemented system · October 2026

1. Epic overview

Epic 3 manages produce **after quality approval**: stock in batches and warehouses, expiry and spoilage monitoring, buyer registration, purchase orders with stock reservation, **auction bidding** as a second sales channel, delivery preparation and tracking, and buyer feedback.

Stakeholders: Inventory Manager, Buyer, Transport Coordinator.

1.1 Story map

Story	What happens	Screen	API	DB
E3-US1 S1	Create / manage inventory batches (created automatically from inspection)	CurrentInventory , BatchDetails	GET /api/inventory , /:id , /:id/adjust	inventory_batches
E3-US2 S1	Warehouses and storage locations	WarehousesPage	/api/warehouses , /inventory/:id/transfer	warehouses, storage_locations
E3-US3 S1	Real-time stock and batch information	InventoryDashboard	GET /api/inventory	inventory_batches, inventory_transactions
E3-US4	Expiry and spoilage tracking	NearExpiryStock , WastageRecords	GET /inventory/expiry , POST /inventory/expiry/sweep , POST /inventory/:id/wastage	record_wastage() , sweep_expired_batches() , wastage_records
E3-US5 S1	Buyer registers and views products	RegisterPage (role buyer), ProductMarketplace	POST /api/buyers , GET /api/prices/current	buyers
E3-US6 S1	Create and track purchase orders	CreateOrder , MyOrders , OrderDetails	POST /api/orders , GET /api/orders	purchase_orders, purchase_order_items
E3-US7 S1	Approve order and reserve stock	OrderDetails (manager view)	PATCH /api/orders/:id/status , POST /:id/allocate	reserve_order_stock() , stock_allocations
E3-US8	Prepare and track deliveries	DeliverySchedule , ActiveDeliveries , VehiclesPage	/api/deliveries	delivery_schedules, transport_vehicles
E3-US9	Prevent overselling, expired allocation, excessive reservation	error messages on approval	inside reserve_order_stock() + trigger	system_settings

Story	What happens	Screen	API	DB
E3-US10	Place a bid	AuctionDetailPage	POST /api/auctions/:id/lots/:lotId/bids	place_auction_bid()
E3-US11	View my bids and statuses	MyBidsPage	GET /api/auctions/my-bids	auction_bids
E3-US12	Inventory manager views and manages bids	AuctionAwardPage , AuctionDashboardPage	GET /api/auctions/:id/award-board	auction_bids, auction_winners
E3-US13	Accept (award) or reject bids	AuctionAwardPage	POST .../award , POST .../offer-next (reject)	award_auction_lot() , offer_to_next_bidder()
E3-US14	Buyer notified of the bid result	NotificationsPage	trigger + notifications	notify_auction_losers()
Epic text: feedback	Buyer rates a delivered order	OrderFeedback on OrderDetails	GET/POST /api/orders/:id/feedback	order_feedback

2. Core concepts

2.1 Batches, stock figures and the stock ledger

- A **batch** = accepted produce from one collection, with grade, quantity, price, expiry, warehouse location and QR value.
- Three numbers matter: **initial**, **available** and **reserved** quantity. Real-time stock = available; reserved stock is held for approved orders.
- Every movement (received, reserved, released, wasted, adjusted, transferred) writes a row to `inventory_transactions` – a **ledger** with a running balance. This gives full history and auditability.
- The stock screen shows stock by crop and grade with a **low-stock flag**, reserved vs available quantity and expiry date.

2.2 FEFO – First-Expiry-First-Out

When an order is approved, `reserve_order_stock()` picks batches ordered by **expiry date ascending** (then received date). The oldest-expiring produce is sold first, which reduces spoilage – exactly the epic's "reduced produce wastage" benefit. Batches are locked with `FOR UPDATE` so two simultaneous orders cannot reserve the same kilos (**concurrency control**).

2.3 Order life-cycle

submitted → under_review → approved / rejected → stock_reserved → payment_pending → paid
 → preparing → dispatched → delivered → completed (cancelled possible before dispatch)

Role	Statuses it may set
Buyer	only cancelled , on their own order, and only before dispatch
Inventory manager	under_review, approved, stock_reserved, rejected, preparing, dispatched, cancelled

Role	Statuses it may set
Finance officer	under_review, payment_pending, paid, rejected, cancelled
Manager	none (read-only role)
Administrator	any (superuser)

Approving an order triggers the atomic stock reservation; cancelling or rejecting calls `release_order_stock()` and the kilos return to available stock. Approved-but-unpaid reservations are **released automatically after 48 hours** (`release_stale_reservations()`, setting `reservation_hold_hours`).

2.4 Expiry and wastage (E3-US4)

- **Expiry overview** lists batches expiring within `near_expiry_days` (default 7). Risk level: **expired** (days left < 0), **critical** (≤ 2 days), **high** ($\leq$ half the window), otherwise **medium**. It also shows kilos and **value at risk** (available kg $\times$ purchase price).
- **Managing near-expiry stock:** for each batch the inventory manager can **change the expiry date** (also used to flag a batch that will not keep), apply a **clearance price** (discount 1–90 %, audited with old and new price) or **write off** part or all of it with a reason. The page also has "Flag a batch as near-expiry" for any batch with stock.
- **Write-off sweep** moves expired batches to wasted stock in one action; **wastage records** store quantity, reason (spoilage, damage, expiry, pest/disease, handling, temperature, other) and the LKR value lost; the Manager's waste analysis uses them.

2.5 Auctions – a second way to sell

```
draft → scheduled → open ⇌ paused → closed → (reserve_not_met | awaiting_award) → awarded
→ payment_pending → paid → stock_allocated → preparing → dispatched → delivered → completed
```

- **Auction dates cannot be in the past:** start and end must be in the future (2-minute grace), and end must be after start – checked in the form and on the server.
- **Automatic opening and closing:** a background scheduler (every 30 seconds, and whenever auctions are listed) opens a *scheduled* auction when its start time arrives and closes an *open* one at its end time, which determines the winners. Without it nobody could bid.
- Lots have a starting price, an optional **reserve price** (minimum acceptable), and a minimum bid increment.
- When the auction closes, the highest bid at or above reserve becomes the **winner offer**; bids below reserve end as *reserve not met*.
- The inventory manager reviews all bids on the **award board: Accept & award** creates a purchase order and invoice; **Reject winning bid** (reason ≥ 10 characters) marks that bid rejected, notifies the bidder and offers the lot to the next highest bidder (or cancels the lot if none).
- Losing bidders are notified automatically when the auction closes (database trigger).

3. Validations

3.1 Buyer registration – E3-US5

- Public registration accepts role `buyer` (or farmer). A buyer then completes their profile: **company name, contact person, email, phone, address, district are all required** (400 "<field> is required").
- **One buyer profile per account** → 409 "A buyer profile already exists for this account". Buyer code is generated by `generate_buyer_code`; duplicate unique values → 409.
- Whitelisted fields only (a buyer cannot set their own verification status or credit limit); staff verify buyers and set the credit limit.

- A buyer sees only their own record; only a **verified, active** buyer may bid.

3.2 Placing an order – E3-US6

Rule	Result
At least one item	400 "At least one crop item is required to place an order"
Each item needs a crop category and a quantity > 0	400
Preferred collection centre required	400
Delivery address required	400
Unit price is never chosen by the buyer: the server looks up the centre's current selling price for the crop and grade (best available grade if none chosen)	prevents price tampering; the order screen shows the price read-only
Buyer record must exist (resolved from the login)	400 "Buyer record could not be found"
Only buyers (or admin) create orders	403 for other roles
A buyer can open only their own orders	403 "You can only access your own orders"

3.3 Approval and reservation – E3-US7 / US9 (inside `reserve_order_stock()`)

Rule	Error
Order must be in <i>submitted</i> or <i>under_review</i>	INVALID_STATE → 409
No overselling: requested kg must not exceed total available kg for that crop and grade	INSUFFICIENT_STOCK → 409 "Tomato requested 500kg but only 300kg is available"
Excessive reservation cap: one order may reserve at most 70 % of the available stock of that item (setting <code>max_reservation_pct_per_order</code>)	EXCESSIVE_RESERVATION → 409
No expired stock: only batches that are available / partially sold and not past expiry are considered	expired batches are skipped; a trigger <code>validate_stock_allocation</code> also raises "Cannot allocate expired batch"
Batch status must not be expired, damaged, disposed or sold	trigger exception
Order row is locked while processing (<code>FOR UPDATE</code>)	prevents two managers approving the same order twice

Inventory staff can also adjust stock: quantity must be a number > 0; reason must be one of the allowed list; adjustment type must be *waste* or *damage*; a write-off cannot exceed the available quantity in the batch (`INVALID_QTY`).

3.4 Order status changes

- Status must be in the allowed list, otherwise 400 "Invalid order status".
- Role rules from the table above → 403 "Your role cannot set an order to ...".
- A finished order cannot change again → 409 "Order is already ... and can no longer change"; buyers cannot cancel after dispatch → 409.

3.5 Warehouses and transfers – E3-US2

- **Creating a warehouse requires a name, a collection centre and an address of at least 5 characters** (400 "Warehouse address is required"); the same rule applies when editing.
- `warehouse_id` is required (400); the warehouse must exist (404) and be **active** (409 "Target warehouse is inactive").

- If a storage location is given it must **belong to that warehouse** (400).
- Only inventory-staff roles may transfer or assign locations.

3.6 Deliveries – E3-US8

- Delivery address and scheduled date are required (400); vehicles need plate number and type; drivers need full name, NIC, phone and licence number.
- A vehicle or driver that is assigned to deliveries **cannot be deleted** (400 "...Consider setting inactive instead").
- Roles are restricted: transport coordinator manages schedules; buyers see **only their own** deliveries; status changes are validated and buyers are notified at each step.

3.7 Bidding – E3-US10 / US11 (inside `place_auction_bid()`)

Check	Code
Lot exists	LOT_NOT_FOUND
Auction is open and the lot is open for bidding	AUCTION_NOT_OPEN / LOT_NOT_OPEN
Bidding window not ended (server time, not the browser clock)	AUCTION_ENDED
Stock batch behind the lot not expired	BATCH_EXPIRED
Bid quantity $\leq$ available lot quantity	INSUFFICIENT_QUANTITY
Minimum bid: at least the starting price, then current price + minimum increment	BID_TOO_LOW
Buyer exists, is verified and the account is active	BUYER_NOT_FOUND / BUYER_NOT_VERIFIED / BUYER_SUSPENDED
Credit limit: bid value must not exceed credit limit minus used credit	CREDIT_LIMIT_EXCEEDED
Only a buyer can bid	403

Bids are placed by an **atomic database function** with a monotonic bid sequence, so two buyers bidding at the same moment are ordered fairly and consistently.

3.8 Award / reject – E3-US12 / US13 / US14

- Only inventory manager, finance officer or administrator can open the award board, award or reject (others → 403).
- **Reject** needs a reason of **at least 10 characters** (422); the database function marks the previous offer as defaulted, cancels its order/invoice and releases reserved stock.
- Award fails if no pending winner exists (404).
- Notifications: winner at close, the next bidder when offered, and the rejected bidder; losers and "reserve not met" are notified by a trigger.

3.9 Buyer feedback

- Only the **buyer who owns the order** can submit; the order must be **delivered or completed** (400).
- Overall rating required; ratings are whole numbers 1–5; comment capped at 1000 characters.
- **One feedback per order** (UNIQUE order_id → 409 "already submitted"); inventory managers are notified; staff see it read-only.

4. Suggested live demo

1. Inventory manager → **Current inventory**: filters, available vs reserved, expiry column (US1, US3). Open a batch to see its QR value and history.
2. **Warehouses**: add a warehouse, transfer a batch; try an inactive warehouse (US2).
3. **Near-expiry** page: risk levels and value at risk; record wastage and show it in **Wastage records** (US4).
4. Buyer → **Marketplace** / prices, **Create order** for more than 70 % of the stock → refused (US9). Then an order within the limit (US6).
5. Manager → open the order → **Approve**: stock is reserved FEFO; show reserved quantity rising (US7). Cancel the order and show stock returning.
6. Transport coordinator → create a delivery schedule, update status; buyer tracks it (US8).
7. Auction: buyer bids below the minimum (rejected), then valid (US10, US11). Manager opens the **award board**, accepts one lot and rejects another with a reason (US12, US13). Show the buyer's notification (US14).
8. After delivery the buyer leaves feedback.

5. Likely viva questions and model answers

Q1. How do you prevent overselling?

Reservation happens inside one database function that locks the batches (`FOR UPDATE`), sums the available stock, and raises `INSUFFICIENT_STOCK` if the order is larger. Because it is transactional, two simultaneous orders cannot take the same kilos.

Q2. What is FEFO and why use it?

First-Expiry-First-Out: reserve from the batch that expires soonest. It minimises spoilage compared with FIFO.

Q3. What is "excessive reservation" and how is it prevented?

One buyer reserving almost all stock would starve others. A configurable cap (default 70 % of available stock for that crop and grade) rejects such orders with `EXCESSIVE_RESERVATION`.

Q4. How do you stop expired stock being sold?

The reservation query skips expired batches, a trigger rejects allocations of expired or damaged batches, and the expiry sweep writes expired stock off as wastage.

Q5. What happens to reserved stock if the buyer never pays?

A scheduled function releases approved-but-unpaid reservations after 48 hours (configurable) and notifies the buyer.

Q6. Why a ledger of inventory transactions?

It records every receive / reserve / release / waste / adjustment with a running balance, so stock can be reconciled and audited.

Q7. How do you ensure fairness in an auction?

Server-side time and a database function with a monotonic bid sequence; minimum increment, reserve price and credit-limit checks are enforced in the same transaction.

Q8. What happens when the manager rejects a winning bid?

The bid is marked rejected with the reason, the bidder is notified, any order, invoice and reservation for it are cancelled, and the lot is offered to the next highest bidder, or cancelled if there is none.

Q9. Can a buyer see another buyer's orders or bids?

No. Order endpoints check ownership (403) and the bids endpoint returns only the caller's bids to buyers; staff views mask bidder identity in public listings.

Q10. Who can change an order's status?

Role rules: buyers can only cancel their own order before dispatch; inventory managers handle approval and dispatch; finance handles payment states; managers are read-only.

Q11. How is delivery tracking done?

A delivery schedule has statuses (scheduled → ... → delivered). The transport coordinator updates it; the buyer sees only their own deliveries and is notified at each change.

Q12. What is the buyer feedback for?

It records satisfaction (overall, quality, delivery) after delivery and is shown to staff, supporting the "buyer feedback" part of the epic.

6. Know your limitations

- Delivery tracking is status-based (no live GPS map).
- Invoices are generated and printable, but no external payment gateway is connected (payments are recorded by finance staff).
- The reservation expiry runs when the release function is called (on a schedule or manually); it is not a continuously running background worker inside the API.
- Automated tests cover the auction rules; reservation and wastage rules are covered by SQL test scripts in `supabase/tests/`, not by browser tests.

7. Where to find it in the code

Concern	File
Stock reservation, release, wastage, expiry sweep	<code>supabase/migrations/024_inventory_integrity.sql</code>
Inventory API	<code>backend/src/controllers/inventoryController.ts</code>
Orders and feedback	<code>backend/src/controllers/orderController.ts</code>
Buyers	<code>backend/src/routes/buyerRoutes.ts</code>
Deliveries and vehicles	<code>backend/src/routes/deliveryRoutes.ts</code>
Auction schema, functions	<code>supabase/migrations/004_auction_schema.sql</code> , <code>006_auction_functions.sql</code> , <code>026_...</code>
Auction API	<code>backend/src/controllers/auctionController.ts</code>
Award board UI	<code>frontend/src/pages/inventory/AuctionAwardPage.tsx</code>
SQL tests	<code>supabase/tests/01_inventory_reservation_wastage.sql</code>

8. Appendix – system foundations (same in every guide)

8.1 Architecture in one picture

```
Browser (React 18 + Vite + Tailwind + TanStack Query + react-hook-form + Zod)
  | HTTPS, JSON, header Authorization: Bearer <JWT>
  v
Express REST API (Node.js + TypeScript)
  helmet · CORS · rate limit · authenticate · requireRole · validate(Zod) · controller · errorHandler
  | supabase-js (service-role key, server side only)
  v
Supabase: PostgreSQL (tables, constraints, RPC functions, triggers) · Auth · Storage (private buckets)
```

Layered design: presentation (React pages) → API/business layer (Express controllers and services) → data layer (Supabase/PostgreSQL). Business rules that must never be bypassed (stock, weights, payments) are enforced in the **database** through PostgreSQL functions, so even a buggy client cannot break them.

8.2 Authentication vs authorisation

Concept	What it answers	How HarvestHub does it
Authentication	"Who are you?"	Email + password are checked by Supabase Auth . On success the API returns a JWT access token and a refresh token. The frontend stores them and the Axios interceptor adds <code>Authorization: Bearer <token></code> to every request; on a 401 it tries the refresh token once, otherwise logs the user out.
Authorisation (RBAC)	"What may you do?"	<code>authenticate</code> middleware verifies the token with Supabase, loads the user's <code>profiles</code> row (role, account_status) and rejects suspended / inactive / pending accounts. <code>requireRole(...)</code> then allows only listed roles (administrator is a superuser). Wrong role → 403 .
Ownership checks	"Is this YOUR record?"	Role alone is not enough: a farmer must not read another farmer's data. Controllers call helpers such as <code>assertFarmerAccess</code> / <code>assertBuyerAccess</code> (services/access.ts) → 403 if the record belongs to someone else.
Row Level Security (RLS)	DB-level safety net	Policies are enabled on tables in Supabase. The API itself uses the service-role key, so the API-layer checks above are the primary control ; RLS protects direct database access.

The nine roles are: **farmer, collection_centre_officer, quality_inspector, inventory_manager, buyer, finance_officer, transport_coordinator, manager, administrator**. Only **farmer** and **buyer** can self-register; staff accounts are created by an administrator.

8.3 Three layers of validation

Layer	Tool	Purpose	Failure looks like
1. Frontend	Zod schema + react-hook-form	Instant feedback, fewer bad requests. Never trusted on its own.	Red message under the field
2. API	Zod schema via <code>validate()</code> middleware + controller rules	The real gate: format, required fields, ownership, state rules.	HTTP 400 with <code>{success:false, message, errors:[{field,message}]}</code>
3. Database	NOT NULL, UNIQUE, CHECK, enums, foreign keys, PL/pgSQL functions	Guarantees integrity even if layers 1–2 are bypassed.	Exception mapped to 404 / 400 / 409

Say this in the evaluation: "We validate on the client for usability, on the server for security, and in the database for integrity. The client is never trusted."

8.4 HTTP status codes used

Code	Meaning	Example in this system
200 / 201	OK / Created	Profile loaded; account or farmer created
400	Bad request – invalid input	Bad NIC format, weak password, quantity ≤ 0
401	Not authenticated	Missing / expired token, wrong password
403	Authenticated but not allowed	Wrong role, or a farmer opening another farmer's record, pending farmer logging in
404	Not found	Unknown farmer / order id
409	Conflict with current state	Duplicate NIC, duplicate booking, invalid status transition, insufficient stock
429	Too many requests	Rate limiter (e.g. crop advisor: 30 requests / 15 min)
500	Server error	Unexpected failure (logged by winston)

8.5 Cross-cutting security measures

- **helmet** security headers; **CORS** restricted to the configured frontend origin in production.
- **Rate limiting** with express-rate-limit (general API and a stricter limit on `/api/auth`).
- **Safe search**: user search text is stripped of characters that could alter a PostgreSQL filter (, () % * \) and limited to 60 characters.
- **Mass-assignment protection**: update endpoints copy only a whitelist of columns (a farmer cannot set their own verification status).
- **Generic login error** ("Invalid email or password") so attackers cannot discover which emails exist.
- **Secrets** only in `.env` (service-role key never reaches the browser); private storage buckets with short-lived signed URLs for documents.
- **Notifications and audit**: important actions create in-app notification rows (and emails when SMTP is configured); auction awards and rejections are written to the audit log.